ESTRUCTURA DE DATOS
SEGFAULT
NULL POINTER
NULL POINTER
ACME
ALGORITMOS
Nodos
Nodos
Binary List
Stack
Pila
Cola
Sorting
Nodos
Binary Search Tree
Recursión
O(n log n)
Stack
Linked List
Sorting
Searching
Big Data
Algoritmos
Algoritmos
IA
IA
O(n log n)
Big Data
Grafo
O(n log n)
ALUMNO: MARVIN IVAN VALDEZ VALDEZ
GRUPO: 209
DOCENTE: CRISTINA MARTINEZ RUIZ

DESCRIPCION

El programa gestiona las sesiones activas de usuarios en el punto PAD-Bienestar del proyecto SIG-Conecta. Cada vez que alguien ocupa un equipo, el sistema registra su número y la hora de inicio. Cuando la persona termina, la sesión desaparece de la lista. En cualquier momento se puede consultar quién está conectado. Está escrito en C y funciona desde la terminal con un menú de cuatro opciones: dar de alta, dar de baja, mostrar sesiones activas y salir.

ESTRUCTURA DE DATOS: LISTA ENLAZADA SIMPLE

Una lista enlazada simple es una cadena de nodos donde cada uno conoce al siguiente, pero no al anterior. En este programa, cada nodo es una sesión de usuario. La lista crece cuando alguien se conecta y encoge cuando alguien se desconecta. No tiene un tamaño fijo: puede tener 0 nodos o 12 al mismo tiempo.

POR QUE MEMORIA DINAMICA Y NO ARREGLOS ESTATICOS.

Un arreglo estático necesita definir su tamaño antes de que el programa arranque. Si declaramos Sesion sesiones [12], los 12 espacios existen, aunque solo haya una sesión activa. Peor aún, si mañana el PAD crece a 15 equipos, habría que recompilar el código.

Con malloc reservamos memoria exactamente cuando la necesitamos, y con free la devolvemos cuando ya no sirve. La lista puede tener cualquier número de nodos sin desperdiciar espacio ni romper límites de compilación.

Otra razón práctica: insertar o eliminar en el medio de un arreglo obliga a mover todos los elementos siguientes. En la lista enlazada, solo se ajustan dos apuntadores y listo

ANALISIS

Explicación técnica del diseño
La estructura Sesion
El núcleo del programa es el struct Sesion. Tiene tres campos:

Campo	Tipo	Descripción
num_equipo	int	Número del equipo (1 al 12). Es la clave de búsqueda.
hora_inicio	char[20]	Cadena con la hora en formato HH:MM:SS, obtenida al registrar.
siguiente	struct Sesion*	Apuntador al siguiente nodo. En el último nodo vale NULL.

Apuntadores utilizados

Apuntador	Función en el programa
cabeza (global)	Apunta al primer nodo de la lista. Si vale NULL, la lista está vacía.
nuevo	Apunta al nodo recién creado con malloc durante el alta
actual	Recorre la lista durante la búsqueda en la baja
anterior	Guarda el nodo previo al que se va a eliminar para reparar el enlace.
aux	Puntero de lectura en mostrar_sesiones y para verificar duplicados
tmp	Auxiliar en liberar_lista para guardar el siguiente antes de liberar

LOGICA DEL SISTEMA

Qué pasa en memoria cuando se registra una sesión Cuando el usuario elige la opción 1 e ingresa un número válido, la función alta_sesion hace lo siguiente:

1. Llama a malloc(sizeof(Sesion)). El sistema operativo reserva exactamente el espacio que ocupa un nodo (un entero, veinte chars y un puntero) y devuelve la dirección de ese bloque. 2. Escribe el número de equipo y la hora actual en los campos del nuevo nodo.
3. Hace que nuevo->siguiente apunte a lo que era la cabeza anterior. Así el nuevo nodo queda al frente de la cadena.
4. Actualiza cabeza = nuevo. La lista ahora empieza en el nodo recién creado.

Qué pasa en memoria cuando se elimina una sesión

La función baja_sesion recorre la lista con dos punteros, actual y anterior, hasta encontrar el nodo del equipo solicitado. Después:

1. Si el nodo a eliminar es la cabeza, mueve cabeza al siguiente. Simple.
2. Si está en medio o al final, hace anterior->siguiente = actual->siguiente. Eso corta al nodo de la cadena sin romper el resto.
3. Llama a free(actual). El bloque de memoria vuelve al sistema y ya no existe en el heap.
4. Pone actual = NULL por buena práctica, para evitar un puntero colgante.

Cómo se recorre la lista para mostrar sesiones

La función mostrar_sesiones usa un puntero aux que arranca en la cabeza. Mientras aux != NULL, imprime los datos del nodo y avanza con aux = aux->siguiente. Cuando llega a NULL significa que terminó la lista.

EVIDENCIA DE EJECUCION

Ejecución con datos correctos
Se registraron cuatro equipos (9, 4, 5 y 3) y se consultó la lista activa. El programa respondió correctamente en todos los casos:

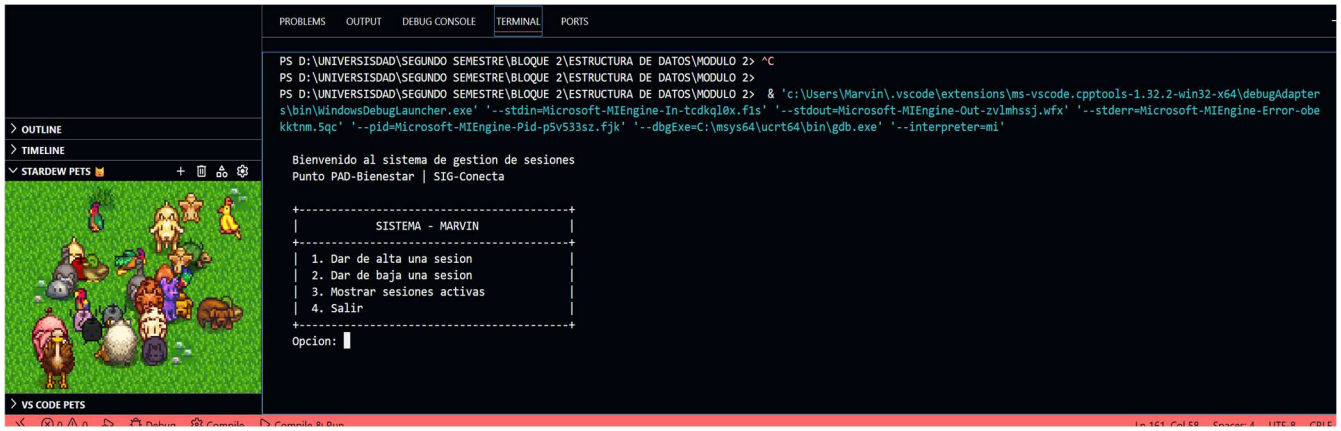
```
+-----------------------------------------+
|              SISTEMA - MARVIN           |
+-----------------------------------------+
|  1. Dar de alta una sesion              |
|  2. Dar de baja una sesion              |
|  3. Mostrar sesiones activas            |
|  4. Salir                               |
+-----------------------------------------+
Opcion: 3

=========================================
SESIONES ACTIVAS - PAD-Bienestar
=========================================
1. Equipo:  9  |  Inicio: 21:00:49
2. Equipo:  4  |  Inicio: 21:00:47
3. Equipo:  5  |  Inicio: 21:00:43
4. Equipo:  3  |  Inicio: 21:00:29
=========================================
Total de sesiones activas: 4
=========================================

+-----------------------------------------+
|              SISTEMA - MARVIN           |
+-----------------------------------------+
|  1. Dar de alta una sesion              |
|  2. Dar de baja una sesion              |
|  3. Mostrar sesiones activas            |
|  4. Salir                               |
+-----------------------------------------+
Opcion:
> Compile & Run
```

Se observa que los equipos aparecen en orden inverso al de registro, porque la inserción se hace al frente de la lista (LIFO). Esto es el comportamiento esperado en una lista enlazada con inserción en cabeza.

Ejecución con datos erróneos
Se probaron cuatro tipos de error: equipo 0 (fuera de rango por abajo), equipo 13 (fuera de rango por arriba), equipo -5 (valor negativo), y dar de alta el mismo equipo dos veces. También se intentó dar de baja con un número fuera de rango (99):

```
+----------------------------------------+
|            SISTEMA - MARVIN            |
+----------------------------------------+
|  1. Dar de alta una sesion             |
|  2. Dar de baja una sesion             |
|  3. Mostrar sesiones activas           |
|  4. Salir                              |
+----------------------------------------+
Opcion: 1

Numero de equipo (1-12): 15
[ERROR] Numero de equipo fuera de rango. Solo se permiten valores del 1 al 12.

+----------------------------------------+
|            SISTEMA - MARVIN            |
+----------------------------------------+
|  1. Dar de alta una sesion             |
|  2. Dar de baja una sesion             |
|  3. Mostrar sesiones activas           |
|  4. Salir                              |
+----------------------------------------+
Opcion: 1

Numero de equipo (1-12): 20
[ERROR] Numero de equipo fuera de rango. Solo se permiten valores del 1 al 12.

+----------------------------------------+
|            SISTEMA - MARVIN            |
+----------------------------------------+
|  1. Dar de alta una sesion             |
|  2. Dar de baja una sesion             |
|  3. Mostrar sesiones activas           |
|  4. Salir                              |
+----------------------------------------+
Opcion:
```

Compilación sin advertencias



Errores técnicos enfrentados y cómo se resolvieron

Puntero colgante al liberar nodo. En la primera versión, después de llamar a free(actual) en la baja, el puntero seguía apuntando a memoria liberada. Se resolvió agregando actual = NULL inmediatamente después del free.

Buffer de stdin sucio con entradas no numéricas.

Cuando el usuario tecleaba una letra en lugar de un número, scanf fallaba y el menú entraba en un bucle infinito leyendo basura del buffer. Se añadió un ciclo while para limpiar el buffer cuando la lectura devuelve un valor distinto de 1.

Inserción en cabeza vs. cola.

Al principio me confundí entre insertar al frente o al final. Elegí la inserción en cabeza porque es O(1) y no requiere recorrer la lista. El orden inverso en la visualización es esperado y no afecta la funcionalidad

Caso de eliminar la cabeza.

La función de baja manejaba bien los nodos del medio, pero crasheaba al intentar eliminar el primero porque anterior valía NULL y se intentaba acceder a anterior->siguiente. Se agregó la condición if (anterior == NULL) para ese caso

Conclusiones

Este programa me permitió entender de manera práctica la diferencia entre tener memoria fija y memoria que crece según la necesidad. Antes de este ejercicio, cuando necesitaba guardar datos pensaba directamente en un arreglo. Ahora sé que eso tiene un costo: si el arreglo es grande, desperdicia memoria; si es pequeño, el programa revienta.

La lista enlazada resuelve ese problema de raíz. Cada sesión existe solo mientras está activa, y desaparece del heap en cuanto el usuario se desconecta. En el contexto del PAD-Bienestar, donde los equipos se ocupan y liberan constantemente, eso es exactamente lo que se necesita. Lo que más trabajo me costó fue entender que hay que mantener dos punteros durante la búsqueda para poder eliminar: uno que avanza y uno que se queda atrás. Sin el puntero anterior, no hay forma de reparar el enlace de la cadena una vez que se libera un nodo.

Comparativo: arreglo estático vs. lista enlazada

Criterio	Arreglo estático	Lista enlazada simple
Tamaño	Fijo en compilación	Variable en tiempo de ejecución
Inserción	O(n) (desplazar elementos)	O(1) (inserción en cabeza)
Eliminación	O(n) (desplazar elementos)	O(n) búsqueda + O(1) borrado
Uso de memoria	Siempre el máximo	Solo la necesaria
Escalabilidad	Requiere recompilar	Sin límite (hasta la RAM disponible)
Acceso directo	O(1) por índice	O(n) — hay que recorrer

La única ventaja real del arreglo estático es el acceso directo por índice (O(1)). Para este sistema eso no importa: nunca necesitamos acceder al equipo número N directamente, siempre buscamos por número de equipo, lo que de todas formas requeriría recorrer el arreglo. Entonces la lista enlazada sale ganando en todos los aspectos relevantes para este problema.

Referencias:

Free Software Foundation. (2022). *Dynamic memory allocation*. En *GNU C Language Manual*. https://gnu-c-language-manual.github.io/GNU-C-Language-Manual/Dynamic-Memory-Allocation.html

Stanford University. (2018). *Linked lists basics* (CS Education Library Document 103). Stanford University. http://cslibrary.stanford.edu/103/LinkedListBasics.pdf

Free Software Foundation. (2022). *Memory allocation*. En *The GNU C Library Reference Manual*. https://man.gnu.org.ua/manpage?3+calloc

Free Software Foundation. (2001). *Memory allocation and C*. En *The GNU C Library Manual* (v. 2.2.5). GNU Project. http://ftp.gnu.org/pub/old-gnu/Manuals/glibc-2.2.5/html_node/Memory-Allocation-and-C.html

Stanford University. (2018). *Pointers and memory* (CS Education Library Document 102). Stanford University. http://cslibrary.stanford.edu/102/PointersAndMemory.pdf